University of Moratuwa
Faculty of Engineering



EN3150 Assignment 03

Simple Convolutional Neural Network to Perform Classification

EN3150 - Pattern Recognition

Group: Mustersucher

Group member	Index number
Devindu W.G.T.	230134U
Wijesekara W.A.G.S.	230724E
Wijesinghe U.G.S.K.D.	230726L
Wijesinghe W.A.P.W.	230727P

Project repository:
github.com/Pamodya-Wijesinghe/Resource-Constrained-CNN-for-Edge-Image-Classification

18 October 2026

List of Acronyms

CNN Convolutional Neural Network

SGD Stochastic Gradient Descent

GPU Graphics Processing Unit

MAC Multiply-Accumulate Operation

RAM Random Access Memory

RGB Red, Green and Blue image channels

SOTA State of the Art

TFLite TensorFlow Lite

ReLU Rectified Linear Unit

Executive Summary

This assignment looks at how a convolutional neural network can classify images when the device has limited memory and processing power. We used the Fashion-MNIST dataset to build two CNNs. Model A uses normal convolution layers, while Model B uses depthwise separable convolutions and a smaller classification part. We then compared them with the pre-trained MobileNetV2 and EfficientNetB0 models.

The training was done in Google Colab using a T4 GPU. We compared Adam, normal SGD and SGD with Momentum. SGD with Momentum gave the lowest final validation loss in our tests, so we used it for both custom models. On the full test set, Model A reached 90.82% accuracy and Model B reached 81.76%. However, Model B needed only 5,539 parameters, compared with 105,866 for Model A. Its saved file and estimated number of operations were also much smaller.

We also tested TensorFlow Lite versions on a Raspberry Pi using 100 images. Model B used the least measured RAM and storage. EfficientNetB0 gave the most correct answers in this small device test, but required much more memory and inference time. Our results show that the smallest model is not always the most accurate or the fastest. The better choice depends on the limits of the device and the accuracy needed for the application.

Contents

List of Acronyms i

Executive Summary ii

1 Introduction 1

1.1 Background 1

1.2 Aim of the Assignment 1

2 Dataset and Experimental Setup 1

2.1 Fashion-MNIST Dataset 1

2.2 Training, Validation and Test Split 2

2.3 Image Preparation 2

2.4 Software and Hardware 3

3 Custom CNN Architecture Design 3

3.1 Model A: Standard CNN 3

3.2 Model B: Lightweight CNN 4

3.3 Activation Functions and Computational Cost 5

4 Optimizer Selection and Tuning 5

4.1 Optimizers Used 5

4.2 Discussion of the Optimizer Results 7

5 Custom Model Training and Evaluation 7

5.1 Training Loss and Validation Loss 7

5.2 Full Test-Set Results 9

5.3 Confusion Matrices and Class-Level Results 9

6 Comparison with Pre-trained Lightweight Models 11

6.1 Models and Fine-Tuning Method 11

6.2 Test Results 11

7 Raspberry Pi Deployment and Testing **13**

7.1 Deployment Method 13

7.2 Measured Device Results 14

7.3 What the Raspberry Pi Results Show 14

8 Final Comparison and Discussion **15**

8.1 Accuracy, Model Size and Operations 15

8.2 Which Model Would We Choose? 15

8.3 Limitations and Possible Improvements 16

9 Conclusion **16**

A Raspberry Pi Evaluation Screenshots **19**

List of Figures

1 Some images from the Fashion-MNIST dataset used for training and testing. . . 2

2 Validation loss for the three optimizer settings over 20 epochs. 6

3 Validation accuracy for the three optimizer settings over 20 epochs. 6

4 Model A training and validation loss over 20 epochs. 7

5 Model B training and validation loss over 20 epochs. 8

6 Confusion matrix for Model A on the full test set. 10

7 Confusion matrix for Model B on the full test set. 10

8 MobileNetV2 confusion matrix for the 10,500-image test set. 12

9 EfficientNetB0 confusion matrix for the 10,500-image test set. 13

10 Recorded Raspberry Pi terminal results for Model A. 19

11 Recorded Raspberry Pi terminal results for Model B. 19

12 Recorded Raspberry Pi terminal results for MobileNetV2. 20

13 Recorded Raspberry Pi terminal results for EfficientNetB0. 20

List of Tables

1 Dataset split used in the experiment. 2

2 Layer details for Model A. 3

3 Layer details for Model B. 4

4 Optimizer results after 20 epochs on the validation set. 5

5 Comparison of the two custom models using 10,500 test images. 9

6 Results of the pre-trained models on the full test set. 11

7 Raspberry Pi results using 100 images for each model. 14

8 Additional inference latency results reported by the device script. 14

9 Overall comparison using the full 10,500-image test set. 15

1 Introduction

1.1 Background

Convolutional Neural Networks (CNNs) are commonly used to identify objects in images. However, not every device has enough memory or processing power to run a large neural network. Small devices such as Raspberry Pi boards and embedded systems often have to work with limited hardware. For these devices, a model must be small enough to store and run, while still giving useful predictions.

A normal CNN can learn image features well, but its convolution and fully connected layers may require many parameters. One way to reduce this cost is to use depthwise separable convolution. It separates spatial filtering from the operation that combines input channels. This can reduce both the number of weights and the number of calculations.

1.2 Aim of the Assignment

The main aim was to build a lightweight image classifier and compare its accuracy and resource requirements against a standard CNN. We also wanted to see whether pre-trained lightweight networks give enough improvement in accuracy to justify their extra memory and computational cost.

For this assignment, we built Model A and Model B, compared three optimizers, trained two pre-trained networks, and measured results on both the full test set and a Raspberry Pi. The tests focus on accuracy, number of parameters, saved model size, estimated operations, inference time, and RAM usage.

2 Dataset and Experimental Setup

2.1 Fashion-MNIST Dataset

We used Fashion-MNIST, which contains 70,000 grayscale images of clothing items in ten classes [1]. Each image is 28×28 pixels, so it already satisfies the assignment limit of 64×64 pixels. Each class has 7,000 images, giving a balanced dataset. The ten classes are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot.

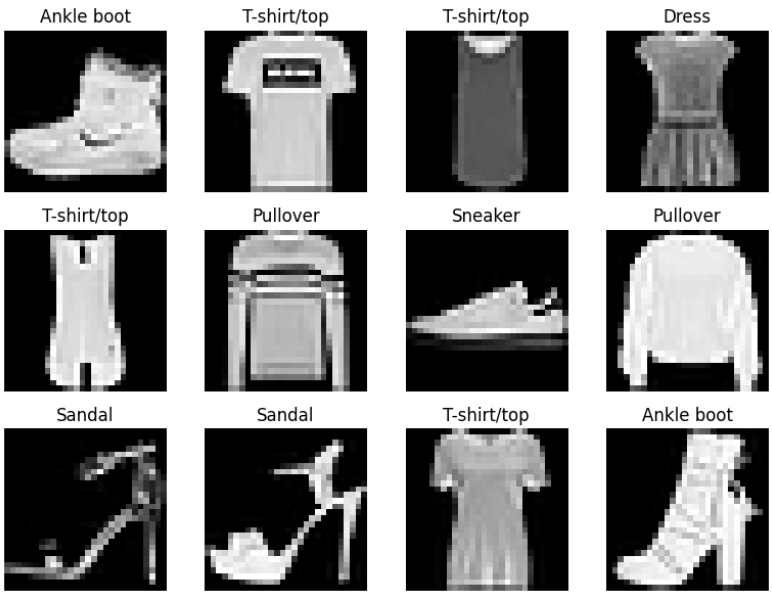


Figure 1: Some images from the Fashion-MNIST dataset used for training and testing.

2.2 Training, Validation and Test Split

We combined the original Fashion-MNIST partitions and made a new stratified split with random state 42. Stratification kept the class distribution the same in each part. The training set was used to update the model weights, the validation set to compare training choices, and the test set for the final results.

Table 1: Dataset split used in the experiment.

Dataset part	Images	Percentage
Training	49,000	70%
Validation	10,500	15%
Testing	10,500	15%
Total	70,000	100%

2.3 Image Preparation

For Model A and Model B, the pixel values were changed from 0 - 255 to 0 - 1 by dividing by 255. The input shape was $28 \times 28 \times 1$, with one grayscale channel. We did not need to downscale these images.

MobileNetV2 and EfficientNetB0 used the same image indices and labels, but their input images were resized to 64×64 and copied into three channels. The model-specific input processing

was then applied: MobileNetV2 used its preprocessing function, while EfficientNetB0 used its built-in preprocessing for the 0 - 255 inputs. Therefore, the dataset split was the same, but the image representation was different for the pre-trained models.

2.4 Software and Hardware

The training experiments were run in Google Colab with an NVIDIA T4 GPU, using Python, TensorFlow/Keras, NumPy, Pandas, Matplotlib and scikit-learn. The notebook uses Python 3.13.15 and TensorFlow 2.21.0. The batch size was 128, and both custom models were trained for 20 epochs. We later tested the TensorFlow Lite versions on a Raspberry Pi CPU.

3 Custom CNN Architecture Design

3.1 Model A: Standard CNN

Model A was built as the baseline. It uses two normal 3×3 convolution layers, each followed by 2×2 max pooling. The number of filters increases from 16 to 32. After pooling, the feature maps are flattened and passed through a dense layer with 64 units and a final layer with 10 outputs.

Table 2: Layer details for Model A.

Layer	Output shape	Parameters
Conv2D, 16 filters, 3×3, ReLU	$28 \times 28 \times 16$	160
MaxPooling2D, 2×2	$14 \times 14 \times 16$	0
Conv2D, 32 filters, 3×3, ReLU	$14 \times 14 \times 32$	4,640
MaxPooling2D, 2×2	$7 \times 7 \times 32$	0
Flatten	1,568	0
Dense, 64 units, ReLU	64	100,416
Dense, 10 units, Softmax	10	650
Total		**105,866**

For a standard convolution, the number of parameters, including biases, is

$$P_{\text{conv}} = (k_h k_w C_{\text{in}} + 1) C_{\text{out}}. \tag{1}$$

The total for Model A is

$$P_A = 160 + 4,640 + 100,416 + 650 = \boxed{105,866}. \tag{2}$$

The largest part comes from the Dense layer after Flatten. This layer alone has 100,416 parameters, which explains why Model A is relatively large even though it has only two convolution layers.

3.2 Model B: Lightweight CNN

Model B was designed to keep the parameter count below 100,000. It uses three 3×3 *depthwise separable* convolution layers, with 16, 32 and 64 output filters. The first two are followed by max pooling. Instead of flattening the last feature maps, we use global average pooling. This gives one value per channel before the final Dense layers of 32 and 10 units.

Table 3: Layer details for Model B.

Layer	Output shape	Parameters
SeparableConv2D, 16 filters, 3×3	$28\times28\times16$	41
MaxPooling2D, 2×2	$14\times14\times16$	0
SeparableConv2D, 32 filters, 3×3	$14\times14\times32$	688
MaxPooling2D, 2×2	$7\times7\times32$	0
SeparableConv2D, 64 filters, 3×3	$7\times7\times64$	2,400
GlobalAveragePooling2D	64	0
Dense, 32 units, ReLU	32	2,080
Dense, 10 units, Softmax	10	330
Total		**5,539**

A separable convolution uses a $k_h \times k_w$ filter for each input channel and a 1×1 operation to combine the channels. With depth multiplier 1, its parameter count is

$$P_{\text{sep}} = k_h k_w C_{\text{in}} + C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{3}$$

The total for Model B is

$$P_B = 41 + 688 + 2,400 + 2,080 + 330 = \boxed{5,539}. \tag{4}$$

This is well below the required 100,000 parameters. Global average pooling is also important here. It replaces a large flattened input to the dense layer with only 64 values. The reduction in Model B is therefore not caused by separable convolutions alone.

3.3 Activation Functions and Computational Cost

We used ReLU for the hidden layers in both custom models because it only needs a comparison with zero. This is simpler than activation functions that need repeated exponential calculations. The final layer uses Softmax because we need probabilities for ten image classes. Softmax is used only at the output, so its cost is limited to ten values.

We also counted the approximate multiply-accumulate operations (MACs) of convolution and dense layers in the notebook. Model A needs 1,117,056 MACs per image, while Model B needs 265,008. That is about a 76.3% reduction for Model B. These are calculated kernel operations, not measured CPU instructions or a complete count of every operation. Pooling, activation functions and some other operations are not included.

4 Optimizer Selection and Tuning

4.1 Optimizers Used

We tested three optimizers using Model B: Adam [4], normal SGD, and SGD with Momentum. All three experiments started with the same initial weights and used the same dataset split, batch size and 20 epochs. Adam used a learning rate of 0.001. Both SGD experiments used 0.01, and momentum was set to 0.9 for the last one. The loss function was sparse categorical cross-entropy.

Table 4: Optimizer results after 20 epochs on the validation set.

Optimizer	Learning rate	Final val. loss	Val. accuracy
Adam	0.001	0.5718	79.02%
SGD	0.010	2.1902	16.38%
SGD + Momentum (0.9)	0.010	**0.4879**	**82.12%**

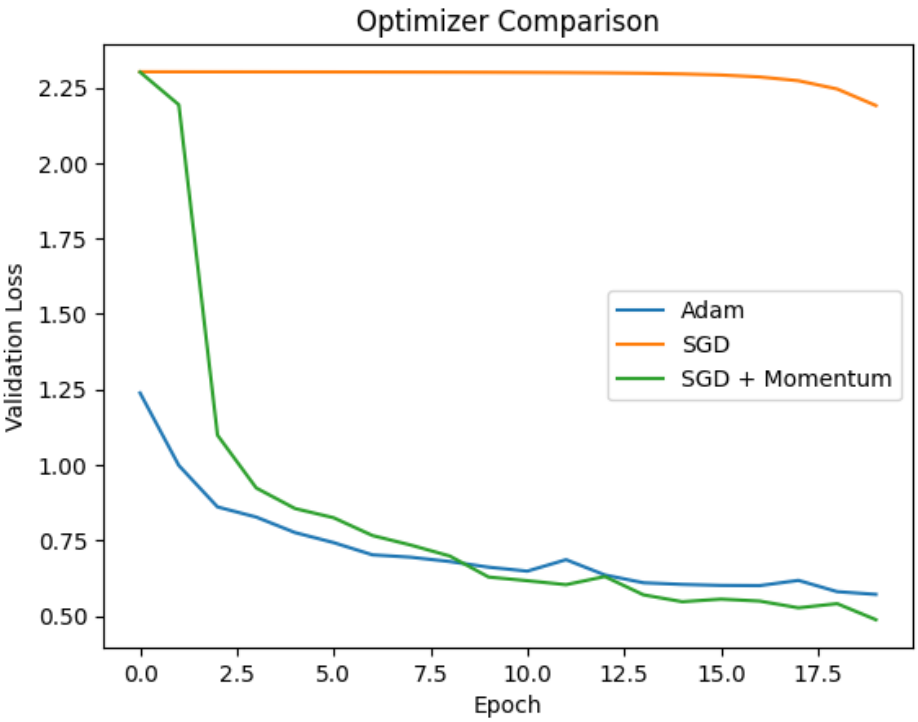


Figure 2: Validation loss for the three optimizer settings over 20 epochs.

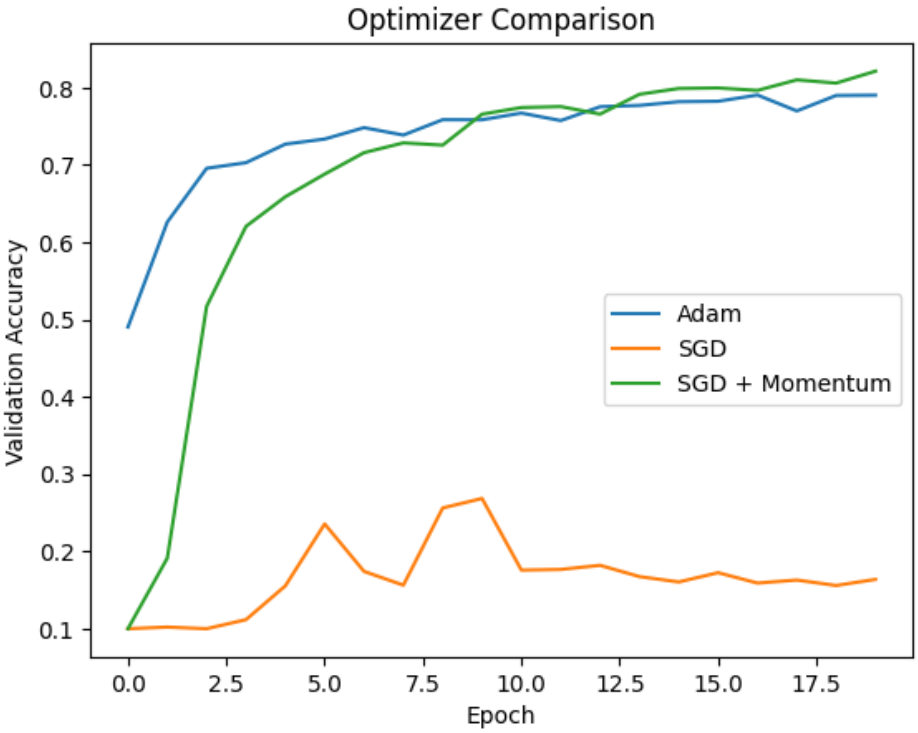


Figure 3: Validation accuracy for the three optimizer settings over 20 epochs.

4.2 Discussion of the Optimizer Results

Adam improved the validation results earlier in training. Normal SGD improved very slowly and still had a high loss after 20 epochs. This means that the chosen learning rate and training time were not enough for normal SGD to learn the task well in this experiment.

SGD with Momentum started slowly, but its loss dropped much faster after the first few epochs. Momentum carries a part of the previous weight update into the next update. This helps the model keep moving in a useful direction instead of changing direction too much between batches. In our experiment, momentum made a clear difference compared with SGD without momentum.

We selected SGD with Momentum because it had the lowest final validation loss, 0.4879. We did not use the test set to select the optimizer. This comparison does not mean momentum will always be better than Adam; we tested only these particular learning rates and one training run for each setting.

5 Custom Model Training and Evaluation

5.1 Training Loss and Validation Loss

After selecting the optimizer, Model A was trained for 20 epochs with SGD and Momentum. For Model B, we used the 20-epoch SGD with Momentum run from the optimizer experiment. Both used batch size 128. We then evaluated the models on the full 10,500-image test set.

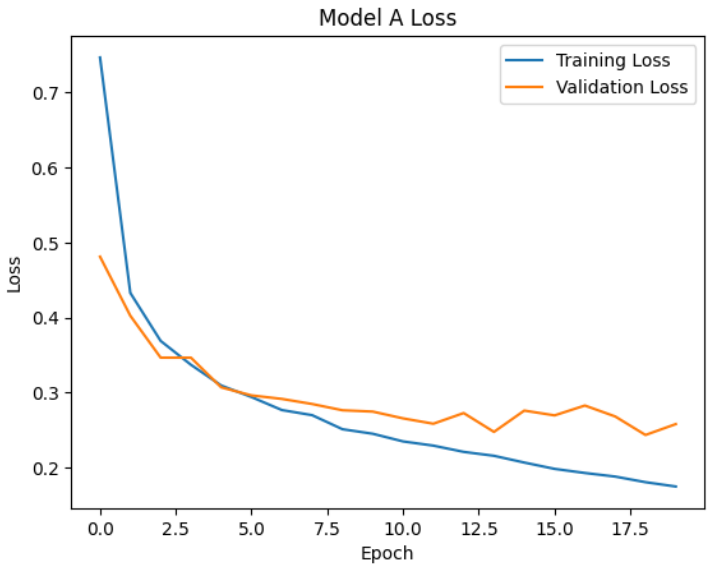


Figure 4: Model A training and validation loss over 20 epochs.

Model A's training loss kept falling through the later epochs, but the validation loss stopped improving as much and moved up and down slightly. This suggests that Model A began to fit the training set better than unseen images. It still gave good test accuracy, but training for more epochs without any changes would not necessarily improve the result.

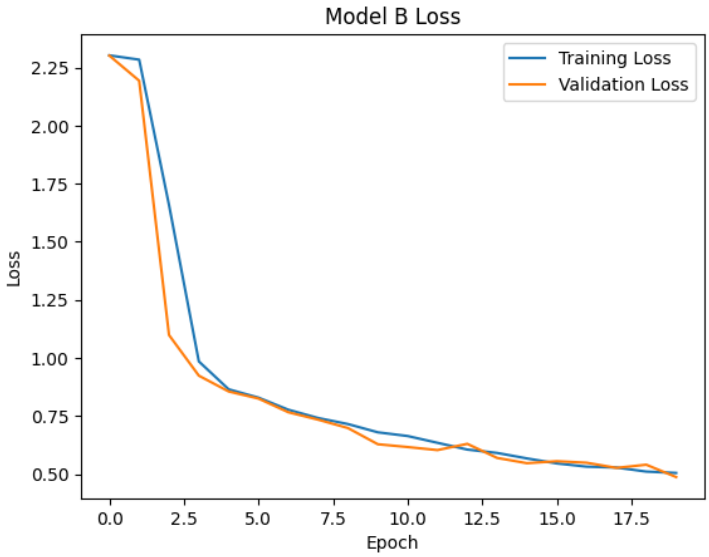


Figure 5: Model B training and validation loss over 20 epochs.

For Model B, both losses decreased and remained much closer to each other. There was no large gap between training and validation loss at the end. However, its final losses were higher than Model A's. This suggests that the smaller model did not learn some of the image differences as well as Model A. A small training-validation gap is not useful by itself if the model is less accurate.

5.2 Full Test-Set Results

Table 5: Comparison of the two custom models using 10,500 test images.

Measurement	Model A	Model B
Number of parameters	105,866	5,539
Saved Keras size (KiB)	861.18	83.77
Mean time per epoch (s)	3.33	3.92
Test accuracy	90.82%	81.76%
Weighted precision	90.96%	81.80%
Weighted recall	90.82%	81.76%
Kernel MACs per image	1,117,056	265,008

Model A correctly classified about 90.82% of the full test images, while Model B reached 81.76%. The difference is around 9.06 percentage points. Model A has more layers' weights available to learn details that may distinguish similar clothing classes. Model B loses some accuracy in exchange for a much smaller network.

Model B reduced the number of parameters by about 94.8% and the saved Keras size by about 90.3%. These are large reductions for a model intended for a device with limited storage. Even so, Model B took about 3.92 seconds per epoch compared with 3.33 seconds for Model A. The reported Colab timings include validation and first-epoch setup. A smaller number of weights or MACs does not guarantee a shorter training time on a GPU, because depthwise operations and other overheads may be handled differently.

5.3 Confusion Matrices and Class-Level Results

Accuracy alone does not show which images are difficult for the model. We therefore looked at the confusion matrices. The rows show the true class and the columns show the predicted class.

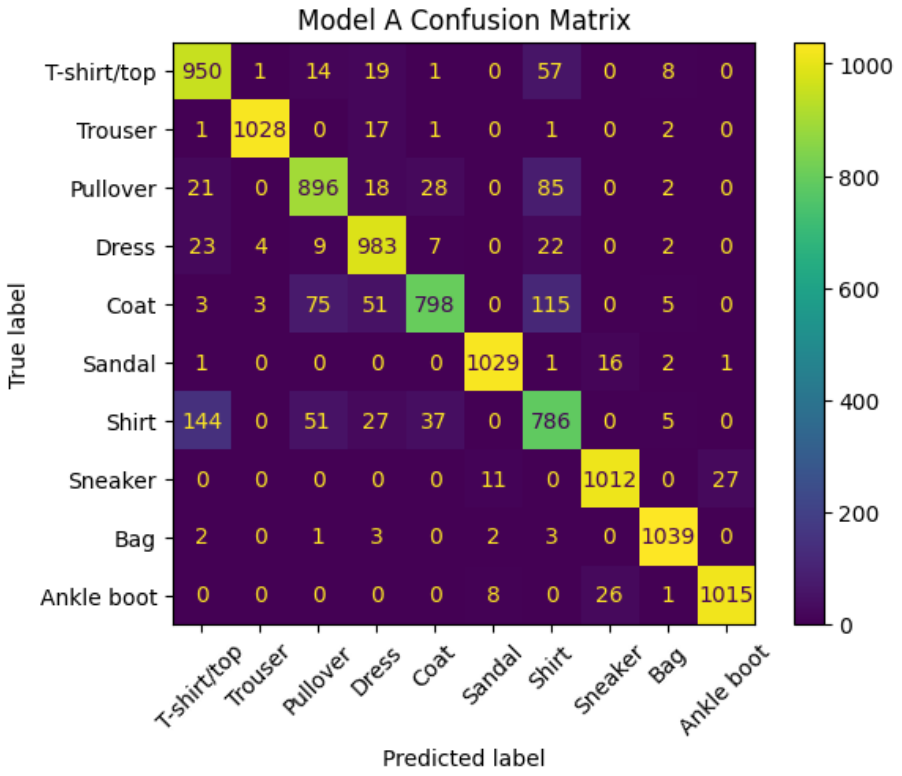


Figure 6: Confusion matrix for Model A on the full test set.

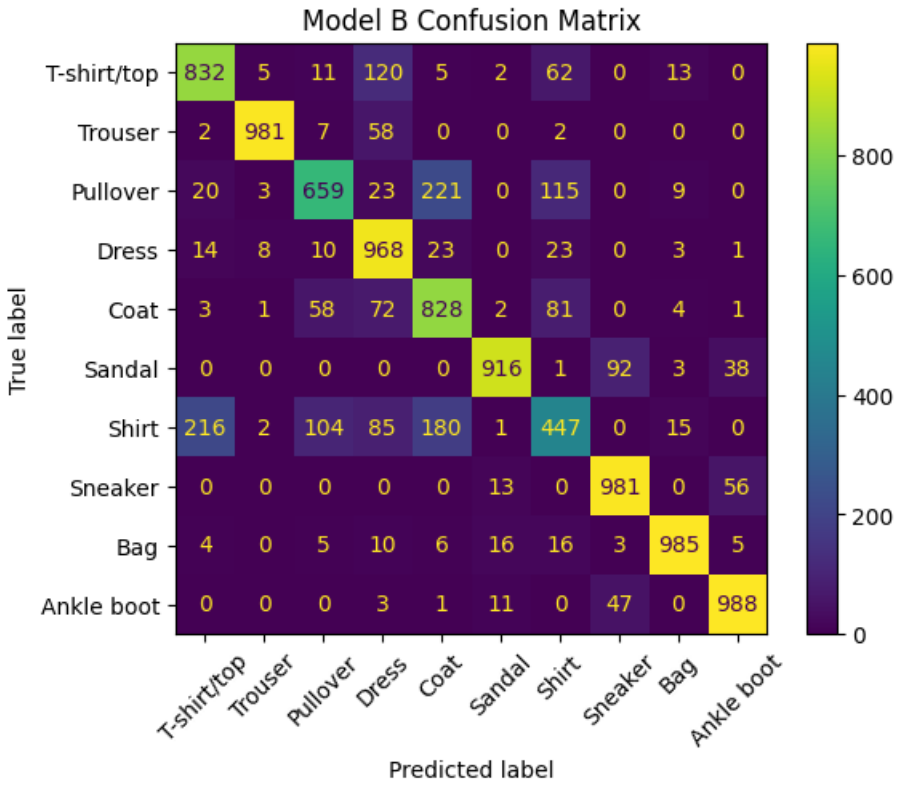


Figure 7: Confusion matrix for Model B on the full test set.

Both models did well for visually different classes such as Trouser, Bag and Ankle boot. Their main mistakes were among upper-body clothes, where shapes can look very similar in a small grayscale image. Model A correctly identified 786 out of 1,050 Shirt images, but Model B identified only 447. This gives Shirt recall values of about 74.86% and 42.57%, respectively.

For Model B, 216 Shirt images were predicted as T-shirt/top, 180 as Coat, and 104 as Pullover. It also confused 221 Pullover images with Coat. These errors help explain its lower accuracy. The model could identify many general clothing shapes, but it had difficulty finding the small differences between similar tops. This matters because the weaker classes are not clear from the overall accuracy number alone.

6 Comparison with Pre-trained Lightweight Models

6.1 Models and Fine-Tuning Method

We selected MobileNetV2 and EfficientNetB0 as the two pre-trained lightweight networks. Both already had weights learned from ImageNet images. We removed their original classifiers and added a new ten-class Softmax output layer for Fashion-MNIST. MobileNetV2 uses mobile-friendly operations [2], while EfficientNetB0 uses an efficient baseline network design [3].

For each model, the original feature-extraction network was first frozen while the new classifier was trained for five epochs with Adam at 0.001. We then allowed the last 20 layers of the original network to be trainable and fine-tuned for another 15 epochs with Adam at 0.00001. The backbone was called in inference mode so that its batch-normalization statistics were not updated. We kept the same training, validation and test partitions as the custom models.

6.2 Test Results

Table 6: Results of the pre-trained models on the full test set.

Measurement	MobileNetV2	EfficientNetB0
Total parameters	2,270,794	4,062,381
Saved Keras size (MiB)	18.54	26.72
Test accuracy	87.99%	89.27%
Weighted precision	88.02%	89.27%
Weighted recall	87.99%	89.27%
Kernel MACs per image	24,461,312	31,979,392

EfficientNetB0 gave the better result of these two models, with 89.27% accuracy. MobileNetV2 reached 87.99%. Both performed better than the 81.76% of Model B. The pre-trained networks can reuse image features learned from a much larger dataset, which is an advantage when adapting to a new classification task.

However, neither pre-trained model beat Model A's full test-set result of 90.82%. Model A was built directly for the 28×28 Fashion-MNIST images, while the pre-trained networks used resized 64×64 RGB copies and a different architecture. Therefore, these tests compare the complete model setups, not only the effect of using pre-trained weights.

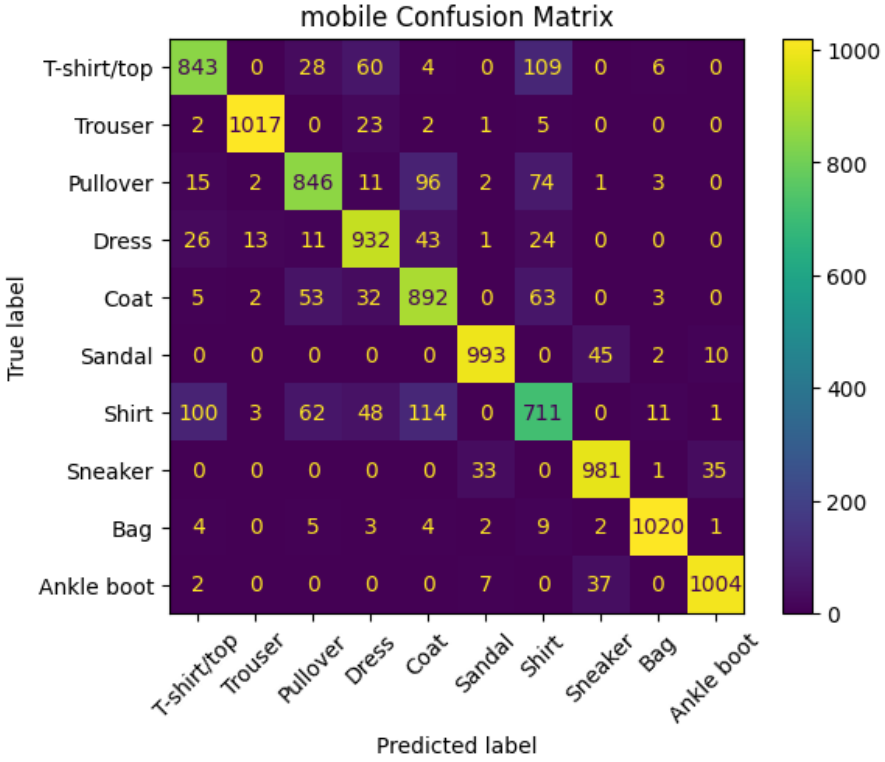


Figure 8: MobileNetV2 confusion matrix for the 10,500-image test set.

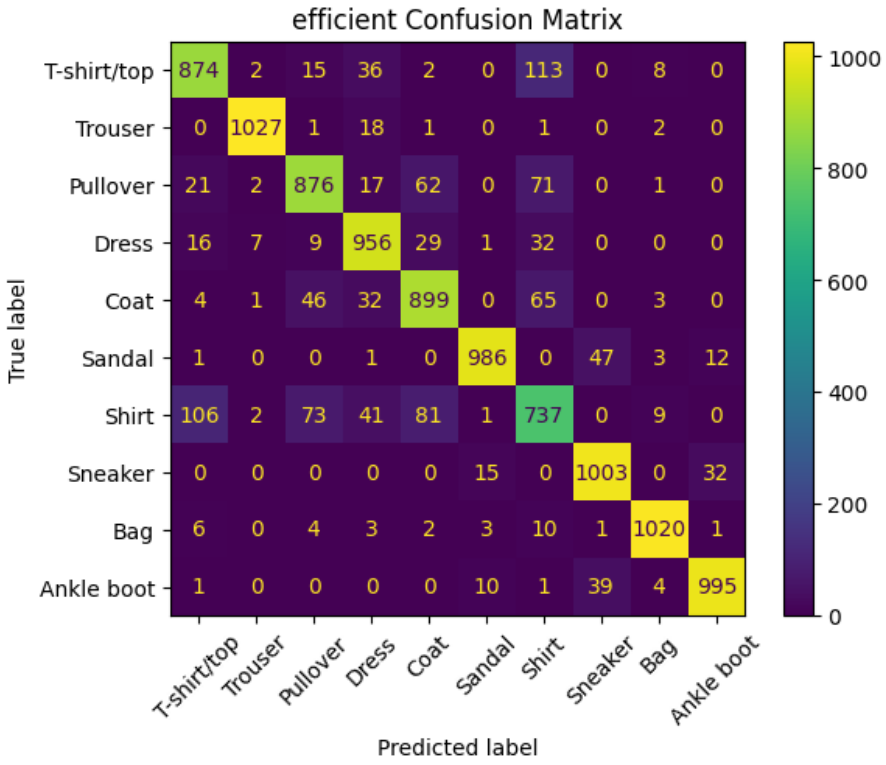


Figure 9: EfficientNetB0 confusion matrix for the 10,500-image test set.

The confusion matrices show that Shirt is still a difficult class for the pre-trained models. MobileNetV2 has about 68% recall for Shirt and EfficientNetB0 has about 70%. This is better than Model B, but it also shows that making a model much larger does not remove every classification problem.

7 Raspberry Pi Deployment and Testing

7.1 Deployment Method

To check how the models behave on a small device, we used TensorFlow Lite versions of the four networks and tested them on a Raspberry Pi CPU. The screenshots in our project show that the evaluation script processed 100 images per model and used the TensorFlow Lite XNNPACK CPU delegate. The custom models received $28 \times 28 \times 1$ inputs, and the pre-trained models received $64 \times 64 \times 3$ inputs. All the screenshots report float32 model inputs.

The device test was separate from the full 10,500-image test evaluation. This is important: an accuracy based on 100 images can change by one percentage point with only one different prediction. We have therefore kept the two sets of results in separate tables.

7.2 Measured Device Results

Table 7: Raspberry Pi results using 100 images for each model.

Model	Correct /100	Mean latency (ms)	Peak RAM (MiB)	TFLite (MB)
Model A	90	0.0575	58.62	0.408
Model B	85	0.0614	40.61	0.027
MobileNetV2	93	1.9515	63.09	8.523
EfficientNetB0	94	4.2518	77.59	15.362

Table 8: Additional inference latency results reported by the device script.

Model	Median (ms)	Minimum (ms)	Maximum (ms)
Model A	0.0417	0.0384	0.2305
Model B	0.0410	0.0327	0.1820
MobileNetV2	1.9036	1.3117	4.0032
EfficientNetB0	4.2270	2.9546	6.9349

7.3 What the Raspberry Pi Results Show

EfficientNetB0 gave the highest accuracy on these 100 images: 94 correct predictions. MobileNetV2 had 93, Model A had 90, and Model B had 85. This is not the same ranking as the full test-set results, where Model A performed best. With only 100 images, the small difference between 93 and 94 correct predictions is not enough to say that one model will always be more accurate on a device.

Model B needed the least reported RAM, 40.61 MiB, and its TensorFlow Lite file was only 0.027 MB. Compared with Model A, its peak RAM was about 30.7% lower and the TFLite file was about 93.4% smaller. This makes Model B attractive when storage and RAM are the main restrictions. The file size and RAM values are different measurements: running a model also needs memory for intermediate tensors and the program itself.

We expected the lower-MAC model to run faster, but Model B's measured mean latency was 0.0614 ms per image, slightly higher than Model A's 0.0575 ms. Its median latency was slightly lower, at 0.0410 ms compared with 0.0417 ms. The readings do not give a clear speed advantage to either custom model. We would need repeated measurements under the same conditions to make a stronger claim. Both custom models were much faster in these measurements than MobileNetV2 and EfficientNetB0.

The original evaluation screenshots do not specify the exact Raspberry Pi model, number of CPU threads, or whether preprocessing is included in the timing. We therefore treat the latency and peak RAM numbers as measurements from the supplied script, rather than as values that will be identical on every Raspberry Pi.

8 Final Comparison and Discussion

8.1 Accuracy, Model Size and Operations

Table 9: Overall comparison using the full 10,500-image test set.

Model	Parameters	Keras size (MiB)	Test accuracy	MACs / image
Model A	105,866	0.841	90.82%	1,117,056
Model B	5,539	0.082	81.76%	265,008
MobileNetV2	2,270,794	18.536	87.99%	24,461,312
EfficientNetB0	4,062,381	26.719	89.27%	31,979,392

Model B is the smallest model by a large margin. It has only 5,539 parameters, compared with more than two million in MobileNetV2 and more than four million in EfficientNetB0. Its estimated kernel operations are also much lower. Model B therefore meets the purpose of making a classifier that is less demanding to store and run.

However, there is an accuracy cost. Compared with Model B, MobileNetV2 improves the full test-set accuracy by about 6.23 percentage points and EfficientNetB0 by about 7.51 percentage points. These improvements come with much larger files and considerably more calculated operations. Model A had the best full test-set accuracy, but it was used as a standard CNN baseline and it exceeds 100,000 parameters. The assignment's parameter limit applies to Model B, which stays comfortably below it.

8.2 Which Model Would We Choose?

If the target device has very little storage and RAM, we would choose Model B. It does not give the highest accuracy, but it takes much less space, has the lowest reported RAM usage on the Raspberry Pi, and stays below the parameter limit. This would be useful for a small edge application where some classification mistakes are acceptable.

If better accuracy matters more than a very small model size, Model A is a reasonable option in this experiment. It achieved the highest full test-set accuracy and very low measured inference

latency, although its parameter count is higher. For a device with more memory, MobileNetV2 may also be worth considering because it improves on Model B's accuracy, without being as large or as slow as EfficientNetB0 in our device test.

EfficientNetB0 used the most storage and RAM and had the highest measured latency. It correctly classified one more of the 100 Raspberry Pi images than MobileNetV2. For this small difference in the device test, its extra resource requirements would not be justified for a strongly limited device without further testing.

8.3 Limitations and Possible Improvements

There are several limitations in our tests. The optimizer comparison used one fixed learning rate per optimizer instead of searching many learning rates. Model B also had trouble classifying similar clothes, especially Shirt. We could try small changes in the number of filters, use data augmentation, or test another small classification head to see if accuracy improves while remaining under 100,000 parameters.

We tested 100 images on the Raspberry Pi. A larger device test and repeated timing runs would give more reliable accuracy and speed comparisons. We could also try post-training quantization and measure whether it reduces RAM and storage further. Since the custom and pre-trained models used different input image sizes, we should keep that difference in mind when comparing computational cost. These would be useful next steps before choosing a model for a real product.

9 Conclusion

The aim of this assignment was to make an image classifier that could work with limited hardware resources. The experiments showed that it is possible to build a very small CNN without losing all useful classification performance. Model B reached 81.76% accuracy on the full Fashion-MNIST test set with only 5,539 parameters. It reduced the parameter count by about 94.8% compared with Model A, although its accuracy was about 9.06 percentage points lower.

The optimizer test also made a difference to the final model. Normal SGD learned slowly with the settings we used. Adam improved faster at the beginning, but SGD with Momentum gave the best final validation loss and was selected for training the two custom models. This showed us that the optimizer and its settings can change the result even when the network architecture stays the same.

The Raspberry Pi results were useful because they showed that calculations on paper do not tell the whole story. Model B had fewer operations and a much smaller file, but its measured mean

inference time was not lower than Model A's. At the same time, it used the least reported RAM and storage. The pre-trained models gave better accuracy than Model B, but they needed much more memory and inference time.

From these results, we would choose Model B when the main requirement is a small model that fits a strict memory budget. If accuracy is more important and the device can support a larger model, Model A or a pre-trained network may be more suitable. We therefore cannot choose the best model by accuracy alone. We need to consider the accuracy, storage, RAM and actual device timing together. This was the main point learned from the assignment.

References

[1] H. Xiao, K. Rasul and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv:1708.07747, 2017.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proceedings of the International Conference on Machine Learning*, 2019.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations*, 2015.

[5] Project source code and saved experiment results, GitHub project repository, accessed October 2026.

A Raspberry Pi Evaluation Screenshots

The images below are cropped from the terminal screenshots saved in our GitHub project. They show the values used for the Raspberry Pi comparison in Section 7. Full screenshots are available in the project repository.

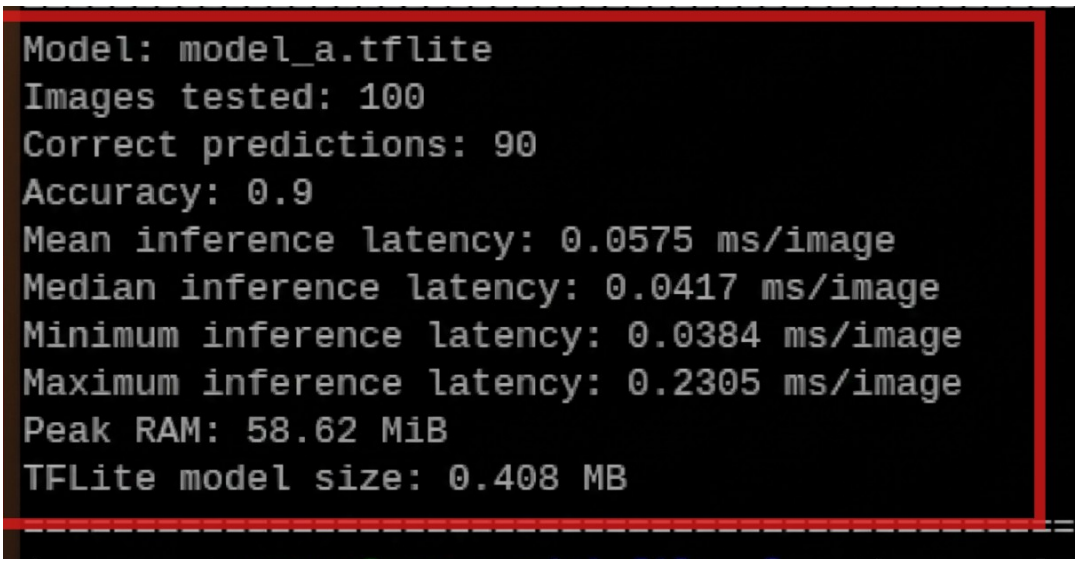


Figure 10: Recorded Raspberry Pi terminal results for Model A.

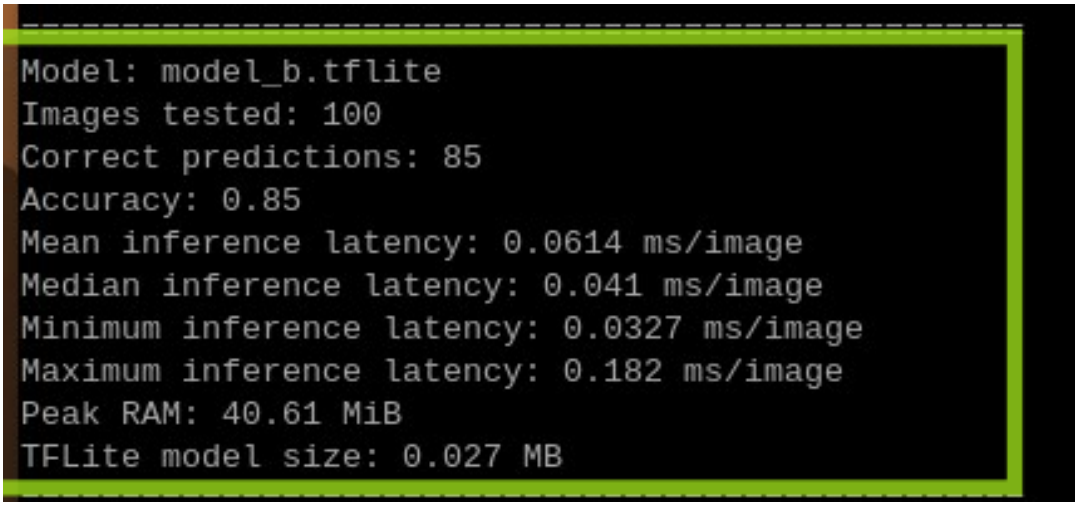


Figure 11: Recorded Raspberry Pi terminal results for Model B.

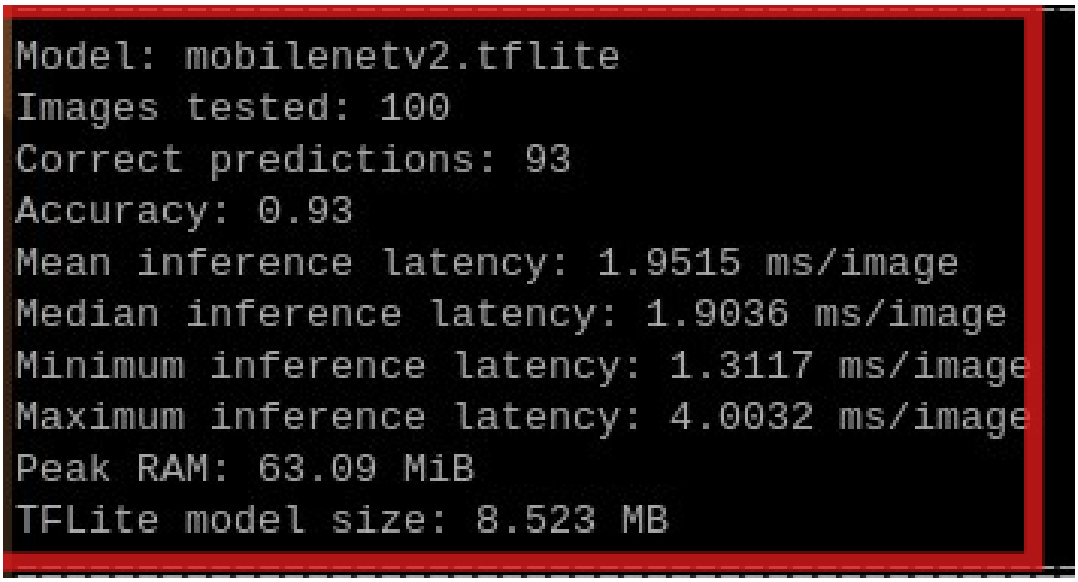


Figure 12: Recorded Raspberry Pi terminal results for MobileNetV2.

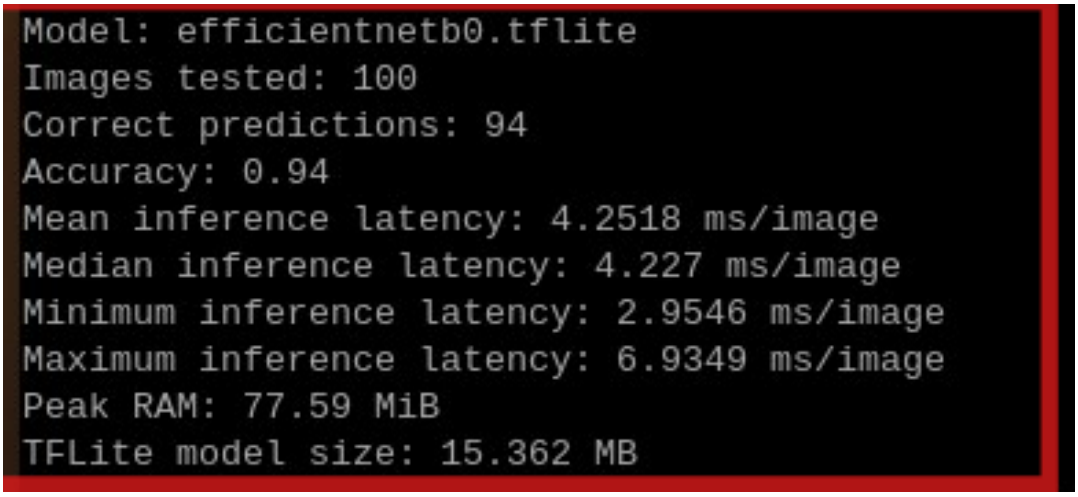


Figure 13: Recorded Raspberry Pi terminal results for EfficientNetB0.